

Using the HmrcVatClient.

Date 03/02/21 (Original)

26/02/21 (Corrections)

Author : Ian H.

version 1.1

The HmrcVatClient component is part of the set of components written to access the HMRC MTD REST API with applications developed in Delphi. It works with both VCL and FMX frameworks. It can be installed on the component palette and dropped on forms at design time, or it can be created dynamically in code.

There are a couple of demo projects that show usage of the VAT clients in all aspects of the MTD VAT process and it is recommended that these are studied. There is also another document, UsingTheDemos, which should be read in conjunction with this one. It covers other aspects of the components and processes.

It is assumed that the reader has studied the VAT API and is aware of the requirements for applications accessing it. This document is intended for competent programmers who just need details of how to use this component; this is not a programming tutorial. Details contained herein were accurate at the time it was written, but the authors cannot be held responsible for any changes that may have taken place since.

The vat client declaration

THmrcVATClient = Class(THMRCRestClient)

Private

Protected

FDateFrom: String; // start date for search

FDateTo: String; // end date for search

Function API_SearchLP(Const aType: String; Const FromDate, ToDate: TDateTime; Var ACorlnId: String): integer;

function CHK_HazNoData(const aCode: integer; Const aValue: TJSONValue): boolean;

procedure INT_Scopes; **override**;

Function PRM_CheckDates(Const dtFrom, dtTo: TDateTime): boolean;

Function PRM_CheckPeriod(Const Value: String): boolean;

Function PRM_CheckValues(Const Values: TStringList): boolean;

Public

Constructor Create(AOwner: TComponent); **Override**;

// search liabilities for a date range

Function GetLiabilities(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String): integer; **overload**;

Function GetLiabilities(Const FromDate, ToDate: TDateTime): TObjectList; **overload**;

// search obligations for a date range

Function GetObligations(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String; aStatus: String = "): integer; **overload**;

Function GetObligations(Const FromDate, ToDate: TDateTime; aStatus: String = "): TObjectList; **overload**;

// search payments for a date range

Function GetPayments(Const FromDate, ToDate: TDateTime; Var ACorrelationId: String): integer; **overload**;

Function GetPayments(Const FromDate, ToDate: TDateTime): TObjectList; **overload**;

```

// do a returns check for a vat period
Function GetReturn(Const aPeriod: String; Var ACorrelationId: String): integer; overload;
Function GetReturn(Const aPeriod: String): TVatReturn; overload;
// submit a return
Function SubmitReturn(Const aPeriod: String; Const Values: TStringList;
    Const IzFinal: boolean; Var Confirm: String): integer; overload;
Function SubmitReturn(aVatObj: TVatSubmission): integer; overload;
// override to apply some kind of validation
Function SetHmrcID(Const Value: String): integer; Override;
End;

```

Ignoring the overridden property method to set the ID, it has 2 public functions to access each of the VAT API endpoints. Except for Submit, for each endpoint, one of the functions returns an integer value, indicating the level of success or failure of the call. If successful, the API response is accessible as a JSON value, otherwise the error code and message are provided. Note that to access the data as JSON requires System.JSON to be declared in the uses clause.

The other function returns either a list of data objects or a single object of the appropriate type. Failure results in a return of nil, when the API error codes are available in the same way. The use of data objects means that the application developers never have to either create or parse JSON directly, if they do not want to.

A note for traditional Delphi database programmers who are new to REST and the internet way of working, where normally a call to load data would just return an empty dataset, if nothing was found, with the API, **no data** results in a 404 Not Found response. No data is treated the same as an invalid request or a processing error.

Declaring and creating a vat component in code.

Uses

..., HmrcRestClient, HmrcRestSupport, HmrcVatClient, HmrcVatSupport, ...

type

TVatForm = class(TForm) // your application form

...

VatClient : THmrcVATClient;

...

procedure SaveToken(Sender: TObject; const aVRN, aScope, aToken, aRefresh: string;
const aExpiry, aTimeOut: TDateTime);

procedure LoadDetails; // load saved application details

procedure LoadTokens(const Value: string); // load any stored access tokens for the current user

...

end;

uses

..., System.JSON, ... // required if not using data objects in methods

procedure TVatFormCreate(Sender: TObject);

begin

VatClient := THmrcVATClient.Create(Self);

VatClient.OnTokenChange := SaveToken; // this should automate token saving

```
// optionally set token load procedure – remove this assignment if not required
VatClient.DoLoadTokens := LoadTokens;

procedure LoadDetails;           // load saved application details
// optionally load all tokens now – remove this call if loading on demand
procedure LoadTokens("");
end;
```

The SaveToken method should match the signature of the THmrcTokenEvent type, declared in HmrcRestSupport. This should save the token details to the chosen data store. The demo projects use an ini file, because it is the simplest to demonstrate, but it can be any storage mechanism. The VatClient component will automatically save new and updated access tokens using this process, if it is set. Otherwise saving tokens would have to be done by some other process. It is assumed that the reader understands OAuth2 and access tokens.

Similarly, saved access tokens need to be loaded by a form/application process and passed to the VatClient. The VatClient provides 2 methods for this, declared in the parent RestClient class:

```
Procedure AddaToken(Const uid, scp, atn, rtn: String; Const exp, tmo: TDateTime);
Procedure SetaToken(Const scp, uid, atn, rtn: String; Const exp, tmo: TDateTime);
```

The load tokens method should match the signature of the THmrcTokenLoad type, declared in HmrcRestSupport, if it is to be used by the client to load dynamically on demand. If an empty string is passed as the parameter, it should load all saved tokens, otherwise it should load only tokens for the supplied UID. In the VAT test demo it is not used and the application loads the tokens after the VRN is set, but since this is something for the application developers to write, it can do whatever they want. It can either be pro-active, where the form triggers the loading, when the UID (VRN) is set, with a form event or it can be reactive and called from the client when needed, although this requires it to be set for the DoLoadTokens event.

In the demo main form, it also initializes some data for the anti-fraud headers in the FormCreate method. This is the data that needs to be set manually, but does not necessarily need to set in this way. See the section below on the headers for more detail.

For VAT, the UID is the VRN. Why is it called UID and not VRN? On the PAYE side, it would be the UTR or the NINo, or something similar.

For successful calls to the API, the JSON response from the API is accessible as [LastValue](#). This is a TJSONValue and can be converted to a string with the ToString call, as shown below with the [VatClient.LastValue.ToString](#) call. Accessing LastValue requires System.JSON to be listed in the **uses** clause.

A call to the Obligations endpoint taken from the demos. Three different ways of calling the Obligations endpoint are shown in the demos, because this endpoint returns data in the sandbox.

```
procedure TVatForm.btnObligationsClick(Sender: TObject);
var
  XCorId: string;
  XResult: integer;
begin
  XCorId := "";
  mmoResponse.Lines.Add("");
```

```

// call the api
XResult := VatClient.GetObligations(dtpFrom.Date, dtpTo.Date, XCorId, Trim(edtStatus.Text));
// and check the result
if (XResult = RESULT_OK) then
begin
    // succeeded, so write data returned to display
    mmoResponse.Lines.Add(VatClient.LastValue.ToString);
    mmoResponse.Lines.Add('Correlation Id : ' + XCorId);
    // or do something more useful with it
    // write the various elements to the screen using the parsing function below
    ParseObligationsJson(VatClient.LastValue);
end
// check whether it failed because there was no data
else if (XResult = RESULT_NODATA) then
begin
    mmoResponse.Lines.Add('The call worked, but no data was found/returned');
end
// check whether it failed because there was no access token
else if (VatClient.LastCode = ERR_NO_ACCESS_TOKEN) then
begin
    // so get a new access token
    VatClient.NewAccessToken;
    // now the user needs to call the api again
    mmoResponse.Lines.Add('Please call the liabilities API again, after authority has been granted');
end
else begin
    // failed, so write error info to display
    mmoResponse.Lines.Add('Error ' + IntToStr(VatClient.LastCode) + ' ' + VatClient.LastMsg);
    mmoResponse.Lines.Add(VatClient.LastError);
    // or handle it some other way ...
end;
end;

```

The process uses the version of the function that returns a status. This format will work for liabilities, obligations and payments, just by changing the client method and parameters. This method handles a couple of possible responses from the API, as explained below.

It starts by declaring 2 variables for the var parameter and result. It then calls the VatClient method with the required parameters and gets the result. If the result is OK, it simply displays the response data to the memo, as before. This is where a real application would do something more useful with the data returned. In this case it also passes the json data to another method in the form, ParseObligationsJson, which shows how this might be handled, if that is what is required.

The Correlation ID is probably of little interest in calls to endpoints other than the vat submissions, but it is provided, so the VatClient picks it up. It is then up to the application developer to either use it or ignore it.

If the result was not OK, then it checks whether the result is No Data. While the API would have returned 404 Not Found, in the cases of Liabilities, Obligations and Payments, it does send a specific error message that can be used to identify it as a no data situation. For other endpoints this no data case cannot be identified, so it is just an error situation.

The next check is the error code. These are defined in the RestSupport unit and generated or passed on by the VatClient component. The no access token error is from the VatClient and means exactly what it says. What the example method does, however, is to detect the error and automatically call the new access token method, which should pop up the authorisation form.

Where there is no existing access token, the liabilities call would have immediately triggered the failed with no access token response from the VatClient and the form would then automatically call the authorisation process. Note the message written after the authorisation call, as it does not automatically recall the liabilities method. The user has to click that button again.

The no access token response should also be triggered in the case where the refresh token has expired after 18 months, so should automatically trigger a fresh authorisation. Note that this is code in the application, handling the responses from the VatClient. The VatClient does automatically refresh the access token after 4 hours, so the application programmer should never have to handle that. There is a more detailed explanation of tokens and refreshing in the other doc.

Finally it gets to displaying the error code and messages, if it was not handled by any of the preceding checks. It is up to the application developer to work out how to handle this.

As stated above, calls to Obligations and Payments can/should be handled in exactly the same way as Liabilities.

A second version of the call to obligations, taken from the FMX demos.

```
procedure TVatForm.btnObligationsClick(Sender: TObject);
var
  ix1: integer;
  XCorId: string;
  XList: TObjectList;
  VatObj: TVatObligation;
begin
  XCorId := '';
  mmoResponse.Lines.Add('');
  if (VatClient.GetObligations(dtpFrom.Date, dtpTo.Date, XCorId, Trim(edtStatus.Text)) =
RESULT_OK) then
  begin
    // succeeded, so write data returned to display
    mmoResponse.Lines.Add(VatClient.LastValue.ToString);
    mmoResponse.Lines.Add('Correlation Id : ' + XCorId);
    // or do something more useful with it
    // now convert the json into a list of vat obligation objects and write their values to the screen
    XList := HmrcVatSupport.ParseObligations(XCorId, VatClient.LastValue);
    if (Assigned(XList)) then
    try
      for ix1 := 0 to XList.Count - 1 do
      begin
        VatObj := TVatObligation(XList[ix1]);
        mmoResponse.Lines.Add(VatObj.Periodkey);
        mmoResponse.Lines.Add(VatObj.XCorId);
        mmoResponse.Lines.Add(VatObj.Status);
        mmoResponse.Lines.Add(DateToStr(VatObj.Start));
        mmoResponse.Lines.Add(DateToStr(VatObj.Stop));
```

```

        mmoResponse.Lines.Add(DateToStr(VatObj.DueBy));
        mmoResponse.Lines.Add(DateToStr(VatObj.Received));
    end; // for
finally
    XList.Free;
end;
end
// check whether it failed because there was no access token
else if (VatClient.LastCode = ERR_NO_ACCESS_TOKEN) then
begin
    // so get a new access token
    VatClient.NewAccessToken;
    // now the user needs to call the api again
    mmoResponse.Lines.Add('Please call the obligations API again, after authority has been
granted');
end
else begin
    // failed, so write error info to display
    mmoResponse.Lines.Add('Error ' + IntToStr(VatClient.LastCode) + ' ' + VatClient.LastMsg);
    mmoResponse.Lines.Add(VatClient.LastError);
    // or handle it some other way ...
end;
end;

```

In this example, the json returned from the endpoint is passed to ParseObligations and converted into a list of vat objects. Here again the data is just displayed on the screen.

The final example, from the VCL demo, uses the alternative method to directly get the list of data objects. This form of call does not have the NoData check option.

```

procedure TForm.VatObligations;
var
    ix1: integer;
    XList: TObjectList;
    VatObj: TVatObligation;
begin
    mmoResponse.Lines.Add("");
    // call the api
    XList := VatClient.GetObligations(dtpStart.Date, dtpEnd.Date, "");
    // and check the result
    if (Assigned(XList)) then
    try
        // do whatever is required with the data
        // just displaying it here, because there is nothing else useful to do
        if (XList.Count > 0) then
            for ix1 := 0 to XList.Count - 1 do
            begin
                VatObj := TVatObligation(XList[ix1]);
                mmoResponse.Lines.Add(VatObj.Periodkey);
                mmoResponse.Lines.Add(VatObj.XCorId);
                mmoResponse.Lines.Add(VatObj.Status);
                mmoResponse.Lines.Add(DateToStr(VatObj.Start));
            end;
        end;
    end;
end;

```

```

        mmoResponse.Lines.Add(DateToStr(VatObj.Stop));
        mmoResponse.Lines.Add(DateToStr(VatObj.DueBy));
        mmoResponse.Lines.Add(DateToStr(VatObj.Received));
    end; // for
finally
    XList.Free;
end
// check whether it failed because there was no access token
else if (VatClient.LastCode = ERR_NO_ACCESS_TOKEN) then
begin
    // so get a new access token
    VatClient.NewAccessToken;
    // now the user needs to call the api again
    mmoResponse.Lines.Add('Please call the obligations API again, after authority has been
granted');
end
else begin
    // failed, so write error info to display
    mmoResponse.Lines.Add('Error ' + IntToStr(VatClient.LastCode) + ' ' + VatClient.LastMsg);
    mmoResponse.Lines.Add(VatClient.LastError);
    // or handle it some other way ...
end;
end;

```

A call to the view returns endpoint taken from the demos

```

procedure TVatForm.btnViewClick(Sender: TObject);
var
    XCorId: string;
begin
    XCorId := '';
    mmoResponse.Lines.Add('');
    if (VatClient.GetReturn(edtViewPeriod.Text, XCorId) = RESULT_OK) then
    begin
        // succeeded, so write data returned to display
        mmoResponse.Lines.Add(VatClient.LastValue.ToString);
        mmoResponse.Lines.Add('Correlation Id : ' + XCorId);
        // or do something more meaningful ...
        // ... obj := HmrcVatSupport.ParseReturns(XCorId, VatClient.LastValue);
    end
    // check whether it failed because there was no access token
    else if (VatClient.LastCode = ERR_NO_ACCESS_TOKEN) then
    begin
        // so get a new access token
        VatClient.NewAccessToken;
        // now the user needs to call the api again
        mmoResponse.Lines.Add('Please call the View Returns API again, after authority has been
granted');
    end
    else begin
        // failed, so write error info to display
        mmoResponse.Lines.Add('Error ' + IntToStr(VatClient.LastCode) + ' ' + VatClient.LastMsg);
    end
end

```

```

    mmoResponse.Lines.Add(VatClient.LastError);
    // or handle it some other way ...
end;
end;

```

So this is just a slight variation on the liabilities call. It does not check for NoData, as it cannot be specifically identified for this endpoint. Everything else is the same. This just leaves the vat submissions endpoint, for which there are 2 overloaded methods in the VatClient. The original method accepts the values in a TStringList, while the second method, introduced in December 2020, accepts the data in a VatReturns object.

The first method accepts 3 input parameters and sets the responses in the string var parameter, which can be parsed later. The second method accepts only the vat object, with all of the submission data and then updates that object with the responses from the endpoint. In both cases the call to the endpoint is the same, it is just the way the data is supplied by and returned to the application that is different.

Below is the newer method that uses the vat returns object, but both are shown in the demos.

```

procedure TVatForm.btnSubmitClick(Sender: TObject);
var
    idx: integer;
    lvText : string;
    XResult: integer;
    obj: TVatSubmission;
begin
    mmoResponse.Lines.Add("");
    // prepare data
    obj := TVatSubmission.Create;
    try
        FillSubmission(obj);
        // submit the return with the list of values
        if (VatClient.SubmitReturn(obj) = RESULT_OK) then
            begin
                // succeeded, so write data returned to display
                mmoResponse.Lines.Add('Correlation Id : ' + obj.XCorId);
                mmoResponse.Lines.Add('Processed : ' + obj.Processed);
                mmoResponse.Lines.Add('Bundle Number : ' + obj.BundleNo);
                mmoResponse.Lines.Add('Payment Indicator : ' + obj.PaymentIdct);
                mmoResponse.Lines.Add('Charge Ref Number : ' + obj.ChargeRefNo);
                mmoResponse.Lines.Add('Receipt Id : ' + obj.ReceiptId);
                // or do something more meaningful ...
            end
            // check whether it failed because there was no access token
        else if (VatClient.LastCode = ERR_NO_ACCESS_TOKEN) then
            begin
                // so get a new access token
                VatClient.NewAccessToken;
                // now the user needs to call the api again
                mmoResponse.Lines.Add('Please call the Submit Return API again, after authority has been
                granted');
            end
        end
    end
end

```

```

else begin
    // failed, so write error info to display
    mmoResponse.Lines.Add('Error ' + IntToStr(VatClient.LastCode) + ' ' + VatClient.LastMsg);
    mmoResponse.Lines.Add(VatClient.LastError);
    // or handle it some other way ...
end;
finally
    obj.Free;
end;
end;
end;

```

Again only a slight variation on the basic format, with failure handled in exactly the same way. It starts by creating a vat object and calling the FillSubmission method. If successful, it writes the data returned by the API to the memo, to show what is returned and how to access it.

In the case of the demo, the fill submission method simply sets object values from text boxes on the demo form. Note that 5 values are as pounds and pence, so floats with 2 decimal places and the other 4 are whole pounds only, so integers. Submitting decimals for these last 4 values will result in an error response from the API.

In a real application the object would be initialized from real data and it is left to the application programmer to work out how to do this in their application. The VAT client does not currently check the format of the data supplied, it just passes it on to the endpoint. It is up to the application developers to ensure that the submitted data is in the appropriate format.

The FillSubmission method from the demos.

```

procedure TVatForm.FillSubmission(aObj: TVatSubmission);
begin
    if (Assigned(aObj)) then
    begin
        aObj.Periodkey := edtSubmitPeriod.Text;
        aObj.DueOnSales := edtDueOnSales.Value;
        aObj.DueOnAcquisitions := edtDueOnAqu.Value;
        aObj.TotalVatDue := edtTotalDue.Value;
        aObj.VatReclaimedCP := edtReclaim.Value;
        aObj.NetVatDue := edtNetDue.Value;
        aObj.SalesExVAT := Trunc(edtSalesEx.Value);
        aObj.PurchasesExVAT := Trunc(edtPurchaseEx.Value);
        aObj.GoodsExVAT := Trunc(edtGoodsEx.Value);
        aObj.AcquisitionsExVAT := Trunc(edtAquEx.Value);
        if (cbxFinalised.ItemIndex = 0) then
            aObj.Finalized := true
        else
            aObj.Finalized := false;
        end;
    end;
end;

```

In the demo, this is an FMX form and the edits are of type TNumberBox, where the value is a float.

That is it for calling the methods to access the API endpoints, but there is still the problem of the anti-fraud headers. For this there are a number of support units to help with the process. Header values should be created, found or calculated by the application and the values passed to the VatClient using the AddaHeader method.

Procedure AddaHeader(Const aName, aValue: String; Const NoEncode: boolean = False);
Procedure RemoveaHeader(Const aName: String);

In the HmrcHeaders unit, there is a method that demonstrates the setting of the headers. This is used in the demos and studying this method and the demo usage should enable the reader to understand the process. In Systematic.VAT.Headers there is an object based approach to the same task and which achieves the same outcome. Both use the same or similar methods to find or calculate that which can be, but both require the application to provide some data.

The demos and support units only show and handle desktop headers and it would be up to the programmer to work out any alternative headers required by other application types.

Some of the header units are or use FMX units. These have been tested in the VCL demos and have been found to work without any issues, although there may be a compiler warning. Similarly, the base rest client contains 1 compiler switch based on a conditional define of FMX, which should be set in FMX projects. This is because the authorisation form is framework specific. In testing, however, the FMX demo worked with the VCL form and the VCL demo worked with the FMX form, but to avoid compiler warnings and potential problems, the conditional define should be used in FMX projects.

The VRN checks are handled by the HmrcTestClient, not the HmrcVatClient. Check the code in the demos.

Some links

The VAT API definitions were here:

<https://developer.service.hmrc.gov.uk/api-documentation/docs/api/service/vat-api/1.0>

The anti-fraud headers were here:

<https://developer.service.hmrc.gov.uk/guides/fraud-prevention/connection-method/desktop-app-direct/>